C#: from first steps to a complete project

Original learning guide | 14 October 2026 | Read, run, change, explain, practice

All commands assume a PowerShell terminal at the repository root. Code is included for study; use the source files for copying and editing.

12 lessons, ordered from easy to hard. Read CSharp-Learning-Guide.pdf for the complete printable course, or use the individual lesson files below.

Start here if programming is new to you. These lessons teach the C# language on .NET 10, including one C# 14 feature in the final project. All examples accept arguments rather than requiring interactive input, so you can repeat and verify them easily.

Before lesson 01

1. Open a PowerShell terminal at the repository root, the folder containing LearningPack.slnx.
2. If local scripts are blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` for this terminal. This changes only this process, not the machine policy.
3. Run `. .\Initialize-Training.ps1` (there is a space between the first and second dots).
4. Keep the terminal at the repository root when copying commands from lessons.
5. For every lesson: predict the output, run it, read each statement, change one thing, complete the exercise, then check SOLUTION.md.

Lesson order

1. 01-Hello: Your first program
2. 02-Types: Variables, types, and numbers
3. 03-Decisions: Input validation and decisions
4. 04-Loops: Loops and arrays
5. 05-Methods: Methods and reusable rules
6. 06-Collections: Lists, dictionaries, and null
7. 07-Classes: Classes, records, and encapsulation
8. 08-Interfaces: Interfaces and composition
9. 09-Linq: LINQ queries and lambdas
10. 10-Files: Files, JSON, and exceptions
11. 11-Async: Async, await, and cancellation
12. 12-Capstone: A persistent study tracker and C# 14

How to know you are ready to move on

Explain the example without reading the notes. Complete the exercise without copying the solution. Make one valid change and one invalid-input experiment. Record what happened in Progress.md. If you cannot explain a result, repeat the lesson with smaller values.

Learn Git with this track

Read the Git workflow and this track's Git checkpoints. Each lesson includes a branch name, verification step, staging paths, and suggested commit. The printable course guide includes the shared Git workflow too.

Official PDF references

The Official-References index lists downloaded publisher PDFs and printable copies of current Microsoft documentation, with source URLs, editions, and course mappings. Publisher PDFs retain their original content. Documentation exports are clearly labeled as local print copies, rather than Microsoft-published PDF books.

Reference documentation

The lesson guides are original teaching material. Official downloads and documentation exports are listed in Official-References/README.md. The links below provide further online study. Checked on 14 October 2026.

- C# guide
- C# 14 features
- LINQ introduction
- Async programming

Course code style

Authored C# follows Google's C# guide, linked from the Google style-guide index.

C# and .NET lessons

- Use two spaces, same-line opening braces, and a 100-column limit.
- Put one statement on each line and brace control-flow bodies.
- Use PascalCase for types/public members, camelCase for parameters/locals, and _camelCase for private fields.
- Place System imports first and sort other imports alphabetically.
- Use block-bodied methods; reserve expression bodies for suitable properties and simple lambdas.
- Put each core type in its own matching source file.
- Use constants for fixed rules and readonly fields for stable dependencies.

.editorconfig records formatter settings and private-field naming. Types now live in files such as StudySession.cs, StudyEntry.cs, and TaskRules.cs. Numbered folders preserve lesson order.

The course still teaches modern C#: top-level programs, file-scoped namespaces, records, collection expressions, and C# 14 field-backed properties. Positional record parameters use PascalCase because they declare public properties.

PowerShell convention

Google's shell guide targets Bash, so these scripts use a documented course convention: two spaces, same-line braces, explicit blocks, and separate statements. Aim for 100 columns; long literal URLs, generated HTML here-strings, and indivisible arguments can exceed that target.

Keep Verb-Noun function names, PascalCase parameters/output properties, and camelCase locals. Use full commands, structured output, help comments, validation, native exit-code checks, and splatting for long calls.

Verify before committing

```
. .\Initialize-Training.ps1
dotnet build .\LearningPack.slnx -m:1 -nr:false
.\Check-Style.ps1
.\Test-LearningPack.ps1
```

To apply the SDK formatter:

```
dotnet format whitespace .\LearningPack.slnx --no-restore
```

```
dotnet format style .\LearningPack.slnx --no-restore --diagnostics IDE0011 IDE0022 IDE0061 --severity
info
```

Review changes and run checks before regenerating course PDFs with Build-Guides.ps1. Original publisher PDFs and official documentation exports retain their source formatting.

Git practice in the CSharp course

Read Git-Workflow.md first and save the supplied baseline before editing. Start each exercise on a clean branch from main. Run focused checks, stage related changes, commit with a clear message, then choose either local integration or a GitHub pull request.

Track checkpoints

Lesson	Branch	Suggested commit	Extra Git skill
01-Hello	feat/csharp-01-city	feat(csharp): accept an optional city argument	Inspect status, diff, and staging
02-Types	feat/csharp-02-discount	feat(csharp): calculate discounted study prices	Inspect status, diff, and staging
03-Decisions	feat/csharp-03-senior-category	feat(csharp): classify senior ages	Inspect status, diff, and staging
04-Loops	feat/csharp-04-session-statistics	feat(csharp): count longer study sessions	Keep commits focused and review the staged snapshot
05-Methods	feat/csharp-05-total-calculation	feat(csharp): validate total calculation inputs	Keep commits focused and review the staged snapshot
06-Collections	feat/csharp-06-topic-totals	feat(csharp): aggregate topic minutes	Keep commits focused and review the staged snapshot
07-Classes	feat/csharp-07-completion-time	feat(csharp): record first completion time	Push a branch and explain validation in a PR
08-Interfaces	feat/csharp-08-memory-notifier	test(csharp): capture notifier messages in memory	Push a branch and explain validation in a PR
09-Linq	feat/csharp-09-topic-summary	feat(csharp): summarize distinct study topics	Push a branch and explain validation in a PR
10-Files	feat/csharp-10-missing-file	fix(csharp): handle missing study files	Review integration history and practice conflict resolution
11-Async	feat/csharp-11-cancellation	test(csharp): verify planning cancellation	Review integration history and practice conflict resolution
12-Capstone	feat/csharp-12-summary-command	feat(csharp): add a per-topic summary command	Review integration history and practice conflict resolution

For a docs-only change, prefer a docs/ branch; for a bug fix, prefer fix/. The suggested feat/ name is a convenient starting point and can be adapted to what you actually implement. Do not describe a documentation-only experiment as a new program feature.

Practice in a separate local repository

```
.\New-GitPractice.ps1 -Topic CSharp
```

Use the printed developer and teammate paths. Both clones and their bare remote are inside .artifacts. Follow the push, merge, and conflict examples from Git-Workflow.md there. This is separate from changing your actual course repository.

A complete checkpoint

1. Verify the change with the lesson commands and an appropriate boundary case.
2. Inspect git diff and stage only related source or notes.
3. Inspect git diff --cached and commit.
4. Push your branch if you intend to publish it.
5. Review a PR or perform a local merge, then verify the behavior on main.
6. Record the resulting commit ID in Progress.md.

Do not commit generated build output, demonstration data, API logs, browser profiles, or credentials. Regenerate the original course PDFs after documentation changes only when the source content is ready; inspect PDF changes by size and rendered pages rather than a text diff.

Git workflow for all three courses

Use Git while completing exercises. Git records versions locally; GitHub hosts a remote repository and adds pull requests. A branch lets you develop a change before integrating it into main. A commit stores the staged snapshot with a message. Push sends commits to a remote. Merge combines histories.

This course uses short-lived branches, small commits, and reviewed merges. It is one practical workflow; team rules can differ. The optional type(scope): summary message convention is a course recommendation, not a Git requirement.

1. Inspect and set your identity

```
git --version
git status --short --branch
git branch --show-current
git remote -v
git log --oneline --decorate -5
```

The working tree is your files on disk. The index, or staging area, holds the next proposed snapshot. HEAD identifies the current commit. origin is a conventional remote name. This repository already has origin: check its actual address instead of adding another remote. Do not run git init inside the lesson folders; all courses belong to one repository.

If identity is missing, replace these placeholders with your own name and verified or GitHub noreply email:

```
git config --local user.name "Your Real Name"
git config --local user.email "your-verified-or-GitHub-noreply-email"
git config --get user.name
git config --get user.email
```

Local identity settings affect only this repository. Commit identity differs from push authentication. Use Git Credential Manager or GitHub's supported authentication; keep passwords and tokens out of source files and remote URLs.

2. Commit the supplied baseline before exercises

An untracked file does not exist in Git history. If the learning pack is initially uncommitted, save its supplied state before editing lessons:

```
git status --short --branch
git switch -c course/learning-pack
git add -- .gitignore .gitattributes .editorconfig README.md STYLE.md Directory.Build.props global.json
NuGet.Config LearningPack.slnx
git add -- Initialize-Training.ps1 Test-LearningPack.ps1 Check-Style.ps1 Build-Guides.ps1
```

```
git add -- Git-Workflow.md New-GitPractice.ps1 Official-Resources.json Download-OfficialReferences.ps1
git add -- CSharp PowerShell DotNet10 .github
git diff --cached --stat
git diff --cached --check
git commit -m "chore(course): add CSharp PowerShell and .NET 10 learning pack"
```

Review every staged path, including deletions. Confirm bin, obj, output, .tools and .artifacts are excluded. Inspect PDF sizes. Add only paths that exist in your copy.

Choose ONE baseline integration path:

- Local practice: git switch main, then git merge --no-ff course/learning-pack -m "Merge initial learning pack".
- GitHub practice: git push -u origin course/learning-pack, open a pull request to main, merge on GitHub, then fetch and fast-forward your local main.

Do not merge locally and also squash the same change on GitHub: those are different histories. If the baseline is already committed on main, skip this section.

3. Start a focused branch

Start with a clean working tree. Commit your current work or deliberately stash it before switching.

```
git switch main
git status --short
# For GitHub practice; omit fetch/pull in local-only practice:
git fetch origin
git pull --ff-only origin main
git switch -c feat/csharp-01-city
```

--ff-only refuses to combine divergent local and remote histories silently. If it refuses, inspect the graph and follow the synchronization section. Names such as feat/csharp-01-city, fix/powershell-09-minute-validation, and feat/dotnet10-task-summary identify one purpose. Branches are repository-wide, not restricted to the named folder. Integrate a lesson branch before starting the next unless you intentionally need a dependent branch.

4. Edit, verify, stage, and commit

For the first C# exercise, implement the optional city argument and verify both explicit and default inputs:

```
. .\Initialize-Training.ps1
dotnet run --project .\CSharp\01-Hello -- Dejan Ljubljana
dotnet run --project .\CSharp\01-Hello -- Dejan
git diff -- CSharp/01-Hello/Program.cs
git add -- CSharp/01-Hello/Program.cs
git diff --cached -- CSharp/01-Hello/Program.cs
git diff --cached --check
git status --short
```

```
git commit -m "feat(csharp): accept an optional city argument" -m "Use Ljubljana when the city is
omitted. Verified explicit and default inputs."
git show --stat HEAD
```

diff shows unstaged edits. diff --cached shows exactly what a commit will contain. Untracked files need editor inspection and staging before they appear in the staged diff. If you edit after staging, add again for edits you want in the same snapshot. git add -p stages selected hunks from tracked files.

Keep each commit focused and working. Use the lesson's verification commands. The root test script checks the supplied behavior; deliberate exercise changes may require updating its expectations.

5. Write clear messages and PR comments

Use an imperative summary describing the resulting change. A roughly 50-72-character summary is a useful readability target, not a Git limit. A body explains why a non-obvious decision was made and what relevant checks passed. Each additional -m adds another paragraph.

Type	Purpose	Example
feat	New behavior	feat(csharp): accept an optional city argument
fix	Correct behavior	fix(powershell): reject negative study minutes
docs	Documentation	docs(dotnet10): explain API validation responses
refactor	Structure with preserved behavior	refactor(csharp): extract study calculations
test	Verification	test(dotnet10): check task persistence
chore	Maintenance	chore(course): normalize source line endings

This is an optional convention. Avoid vague messages such as updates, fix stuff, or final. Source comments explain implementation details; commit messages explain a recorded change; PR comments discuss a proposed change and review. They have different purposes.

For a PR, state the problem, resulting behavior, and exact validation. The supplied .github/PULL_REQUEST_TEMPLATE.md provides those fields. Example title: Accept an optional city in the greeting. Example description: The greeting accepts a second argument and defaults to Ljubljana. Verified explicit and omitted city runs.

6. Push and review a pull request

```
git remote -v
git push -u origin feat/csharp-01-city
```

-u records the upstream; subsequent git push uses that relationship. Push publishes commits but does not merge main. On GitHub, create a pull request with main as base and your feature branch as head. Review Files changed, fix failed checks, and respond to comments. Additional pushes to that branch update the PR.

If a push is rejected, read the error: authentication, branch protection, and remote changes require different fixes. Fetch and inspect before attempting any history rewrite.

7. Merge: choose one completion path

Method	History effect	Useful context
Fast-forward	Moves the target pointer to a descendant	Simple local exercises
Merge commit	Records two parent histories	Preserve individual commits and integration point
Squash merge	Creates one new combined commit	One compact reviewed feature
Rebase then merge	Replays commits on a new base	Teams requiring linear history

Use a merge commit in the local lab to see branch history. For a small GitHub PR, squash merge is a reasonable default if repository policy allows it. Review the final squash message because it becomes main's commit message.

Local completion, without publishing:

```
git switch main
git status --short
git merge --no-ff feat/csharp-01-city -m "Merge CSharp city argument exercise"
git log --oneline --graph --decorate -10
git branch -d feat/csharp-01-city
```

Merge changes your CURRENT branch. --no-ff requests an explicit merge commit. Delete a branch only after integration. -d checks ancestry. After a GitHub squash merge, -d may refuse because the original commits were not merged by identity; keeping the branch is fine.

GitHub completion: merge the PR on GitHub first, then:

```
git fetch origin
git switch main
git pull --ff-only origin main
git log --oneline --graph --decorate -10
git status --short
```

Check the completed behavior is on main before starting the next lesson. git fetch --prune removes stale remote-tracking names, not your local branches.

To incorporate updated main into an active feature branch:

```
git switch feat/csharp-01-city
git fetch origin
git merge origin/main
# Resolve conflicts if needed, verify, and finish the merge.
git push
```

Rebase is another choice for a private, never-pushed branch. It changes commit IDs. This course uses merge to update published branches without rewriting their shared history.

8. Resolve conflicts and undo deliberately

A conflict means overlapping changes need your decision. Run status, open the file, find <<<<<<<, =======, and >>>>>>>, write the intended combined result, remove the markers, and verify it. Stage each resolved file and commit:

```
git status
# Edit and run checks BEFORE staging:
git add -- CSharp/01-Hello/Program.cs
git diff --cached
git commit -m "merge(csharp): preserve city input and updated greeting"
```

git merge --abort abandons the merge where possible. Start merges with a clean working tree so the pre-merge state is clear. Practice your first conflict in the isolated lab.

Command	Effect	Appropriate use
git restore --staged -- path	Unstage while keeping working edits	Wrong path staged
git restore -- path	Discard unstaged tracked-file edits	Only after reviewing and choosing to discard
git stash push -u -m "lesson draft"	Park edits and untracked files locally	Switch context temporarily
git stash list	Show parked work	Find your draft
git stash apply	Restore the latest stash while keeping it	Resume, inspect, then drop after verification
git revert commit-id	Record the inverse of a normal commit	Undo shared work without rewriting history

Stash is local temporary storage, not a pushed backup. Reverting a merge requires additional parent selection; the simple revert example is for a normal commit. Amend rewrites the latest commit; reserve it for appropriate private, unpushed work. Routine beginner work does not require reset --hard, clean -fd, or force pushes.

9. Practice pushing and conflicts offline

New-GitPractice.ps1 creates a fresh developer repository, bare local remote, and teammate clone under .artifacts. It copies your selected topic's first runnable lesson and configures fictional identities only inside those lab repositories. It does not change the course repository's branch or contact GitHub.

```
.\New-GitPractice.ps1 -Topic CSharp
# Use the developer path printed by the script:
Set-Location 'PASTE-THE-PRINTED-DEVELOPER-FOLDER'
git switch -c feat/practice-note
Add-Content -LiteralPath PracticeNotes.txt -Value 'I can explain the staging area.'
git diff
git add -- PracticeNotes.txt
git diff --cached
git commit -m "docs(csharp): record staging practice"
git push -u origin feat/practice-note
```

```
git switch main
git merge --no-ff feat/practice-note -m "Merge staging practice"
git push origin main
git log --oneline --graph --all
```

Open another terminal in the printed teammate folder. Run git pull --ff-only origin main and inspect PracticeNotes.txt. This demonstrates the same data movement without publishing to your GitHub account.

In the lab developer folder, create a deliberate conflict:

```
git switch main
git switch -c feat/conflict-note
Set-Content -LiteralPath PracticeNotes.txt -Value 'Feature: validate inputs before saving.'
git add -- PracticeNotes.txt
git commit -m "docs(course): describe validation"
git switch main
Set-Content -LiteralPath PracticeNotes.txt -Value 'Main: verify persistence after saving.'
git add -- PracticeNotes.txt
git commit -m "docs(course): describe persistence"
git merge feat/conflict-note
# EXPECT a conflict and nonzero exit code. Resolve the file:
git status
Get-Content -LiteralPath PracticeNotes.txt
Set-Content -LiteralPath PracticeNotes.txt -Value 'Validate inputs before saving, then verify
persistence.'
git add -- PracticeNotes.txt
git commit -m "merge(course): combine validation and persistence notes"
git log --oneline --graph --all
```

Both branches replaced the same text differently. Your resolution preserves both ideas. Return to the original repository root when finished.

10. Apply Git in every course

Each lesson contains a Git checkpoint with a branch name, focused verification, staging paths, and suggested message. Git-Practice.md maps your track. Use lessons 01-03 for status, branches, and staging; middle lessons for focused commits and review; final lessons for PRs, integration, and conflict practice.

For a combined API/client change, one focused branch can include both DotNet10 and PowerShell files because they implement one contract. Include only related changes. Keep generated files and secrets out of history.

Official references

This is original instructional material. Further detail comes from Git's command manuals and GitHub's workflow guide:

- git status
- git add

- git commit
- git merge
- git push
- git fetch
- git rebase
- GitHub flow

CSharp: official reference PDFs

This folder contains **5 PDFs** from official source material. Files labeled Publisher PDF are the original downloaded publications. Files labeled Documentation export are locally printed copies of official Microsoft Learn articles, with source links, attribution, retrieval dates, and print formatting. They are not Microsoft-published PDF books or complete product manuals.

Read in this order

Use the course's original learning guide and runnable lessons first. Then consult the relevant current Microsoft documentation article for a deeper explanation. Read standards and architecture books when you want formal language/runtime details or further architecture study.

ECMA-334's seventh edition specifies C# 7. It does not cover C# 14. Use Microsoft-CSharp14-Features.pdf for the newer language features used in the capstone.

PDF catalog

PDF	Kind	Read with course lessons	Scope / edition
ECMA-334-CSharp-7th-Edition.pdf	Publisher PDF	All lessons; advanced reference	C# language standard, 7th edition (December 2023), covering C# 7. Not a C# 14 specification.
Microsoft-CSharp-Overview.pdf	Documentation export	01-08	Current Microsoft Learn C# overview.
Microsoft-CSharp14-Features.pdf	Documentation export	12	Microsoft Learn C# 14 feature overview.
Microsoft-LINQ.pdf	Documentation export	09	Microsoft Learn introduction to LINQ queries.
Microsoft-Async-Await.pdf	Documentation export	11	Microsoft Learn asynchronous programming guidance.

Sources

- ECMA-334-CSharp-7th-Edition.pdf: official source and PDF download.
- Microsoft-CSharp-Overview.pdf: official source.
- Microsoft-CSharp14-Features.pdf: official source.
- Microsoft-LINQ.pdf: official source.
- Microsoft-Async-Await.pdf: official source.

Provenance and refresh

Download-Record.json records the source URL, retrieval timestamp in UTC, file size, PDF SHA-256 hash, and original HTML hash for each documentation export. The retrieval timestamp records when the copy was collected; it is not the document's publication date. Article metadata dates are preserved where supplied. The official article remains the source of truth for changes after this snapshot.

From the repository root:

```
.\Download-OfficialReferences.ps1
```

This refreshes the collection using Official-Resources.json. It requires internet access and local Chrome or Edge. It downloads only the public documents and creates local print copies; it does not install software or publish anything.

Attribution

Original publisher PDFs retain their own copyright, license notices, and source formatting. Ecma standards include their copying terms. Microsoft Learn article wording and examples are retained while navigation is removed and print styling and a provenance banner are added. Credit belongs to Microsoft and the documentation contributors, including named authors credited in the source. See the Microsoft Learn terms and the .NET documentation CC BY 4.0 license. This course and its exports do not imply publisher endorsement.

01-Hello: Your first program

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn output and command-line arguments. Complete the setup in the track README.

Understand the idea

Console.WriteLine writes one line to the terminal. A string is text inside double quotes. A semicolon ends a statement. Top-level statements let the compiler create the entry point for you. args is an array of text arguments; Length tells you how many items it contains. Arrays count from zero, so args[0] is the first argument. The conditional expression condition ? yes : no chooses one value. A dollar sign before a string enables interpolation: the value inside braces is inserted into the text.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\01-Hello -- Dejan
```

Expected result

```
Hello, C#!
Welcome, Dejan!
```

Exercise

Add a second argument for your city. If it is missing, use Ljubljana. Print a third line: City: Ljubljana.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Check args.Length > 1 before reading args[1].

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-01-city.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-01-city
# Implement the exercise, then verify:
dotnet run --project .\CSharp\01-Hello -- Dejan Ljubljana
git diff -- CSharp/01-Hello/Program.cs
git add -- CSharp/01-Hello/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): accept an optional city argument"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-01-city`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.01.Hello.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
```

```
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
// Top-level statements are the entry point of this application.
Console.WriteLine("Hello, C#!");
string name = args.Length > 0 ? args[0] : "Learner";
Console.WriteLine($"Welcome, {name}!");
```

01-Hello exercise solution

Add a second argument for your city. If it is missing, use Ljubljana. Print a third line: City: Ljubljana.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
string city = args.Length > 1 ? args[1] : "Ljubljana";
Console.WriteLine($"City: {city}");
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

02-Types: Variables, types, and numbers

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn types, arithmetic, and formatting. Complete 01-Hello first.

Understand the idea

A variable has a name, a type, and a value. int stores whole numbers; decimal stores base-10 values useful for money; double stores approximate floating-point values; bool stores true or false. The m suffix makes a numeric literal a decimal. A cast such as (double)completed converts a value explicitly. Dividing two integers truncates the fractional part, so 3 / 12 gives zero; convert one operand before dividing. F2 prints two decimal places. CultureInfo.InvariantCulture makes the decimal separator predictable. var would ask the compiler to infer a type; it does not make the variable dynamically typed.

Step by step

The fixed sample values use const because they are known at compile time. A constant cannot be reassigned; an ordinary variable can. Later lessons use mutable counters and collections when values need to change.

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\02-Types
```

Expected result

```
C#: 3 lessons; beginner=True
Total: 25.00 EUR
Progress: 25 %
```

Exercise

Calculate the price after a 20% discount and print 10.00 EUR.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

A 20% discount leaves 80% of the original amount.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-02-discount.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-02-discount
# Implement the exercise, then verify:
dotnet run --project .\CSharp\02-Types
git diff -- CSharp/02-Types/Program.cs
git add -- CSharp/02-Types/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): calculate discounted study prices"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-02-discount`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.02.Types.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
using System.Globalization;

// Use decimal for monetary examples, and invariant formatting for predictable output.
const string course = "C#";
const int completed = 3;
const decimal price = 12.50m;
const bool isBeginner = true;
const double ratio = (double)completed / 12;
Console.WriteLine($"{course}: {completed} lessons; beginner={isBeginner}");
Console.WriteLine($"Total: {(price * 2).ToString("F2", CultureInfo.InvariantCulture)} EUR");
Console.WriteLine($"Progress: {ratio.ToString("P0", CultureInfo.InvariantCulture)}");
```

02-Types exercise solution

Calculate the price after a 20% discount and print 10.00 EUR.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
decimal discounted = price * 0.80m;
Console.WriteLine($"{discounted.ToString("F2", CultureInfo.InvariantCulture)} EUR");
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

03-Decisions: Input validation and decisions

Level: Beginner | Suggested time: 30-45 minutes

Goal

Learn parsing, boolean logic, if, and switch. Complete 02-Types first.

Understand the idea

Text from a terminal is not automatically a number. int.TryParse attempts a conversion and returns a Boolean success value. out int age creates a variable for the parsed result. The ! operator negates a Boolean. || means or and short-circuits: later conditions are evaluated only when needed. An if block executes only when its condition is true. return exits the top-level program early. A switch expression selects a result based on patterns. The underscore is the fallback pattern. Validating values before using them keeps incorrect input away from the rest of the program.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\03-Decisions -- 18
```

Expected result

```
Age 18: adult
```

Exercise

Add a senior category for ages 65 and above. Check ages 12, 13, 17, 18, 64, 65, -1 and abc.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Place the >= 65 pattern before the fallback.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-03-senior-category.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-03-senior-category
# Implement the exercise, then verify:
dotnet run --project .\CSharp\03-Decisions
git diff -- CSharp/03-Decisions/Program.cs
git add -- CSharp/03-Decisions/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): classify senior ages"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Explain the difference between working edits and the staged snapshot.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-03-senior-category`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.03.Decisions.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
```

```
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
// Accept input as an argument so this lesson also runs without a prompt.
string input = args.Length > 0 ? args[0] : "18";
if (!int.TryParse(input, out int age) || age < 0 || age > 130) {
  Console.WriteLine("Enter a whole-number age between 0 and 130.");
  return;
}
string category = age switch {
  < 13 => "child",
  < 18 => "teenager",
  _ => "adult"
};
Console.WriteLine($"Age {age}: {category}");
```

03-Decisions exercise solution

Add a senior category for ages 65 and above. Check ages 12, 13, 17, 18, 64, 65, -1 and abc.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
string category = age switch {
  < 13 => "child",
  < 18 => "teenager",
  >= 65 => "senior",
  _ => "adult"
};
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

04-Loops: Loops and arrays

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn repeated work and zero-based indexing. Complete 03-Decisions first.

Understand the idea

An array groups values of one type. The square-bracket collection expression creates the array. A for loop has initialization, a condition, and an update. index starts at zero; index < Length keeps access inside the array. index++ increases the counter by one. total += value is shorthand for adding a value and assigning the result. A while loop tests its condition before each iteration. Its body must eventually change the condition, or it can run forever. A foreach loop is useful when you need each value but not its index. Reading beyond the array's last index throws an exception.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\04-Loops
```

Expected result

```
Session 1: 15 minutes
Session 2: 20 minutes
Session 3: 25 minutes
Total: 60 minutes
Practice rounds left: 2
Practice rounds left: 1
```

Exercise

Print the average session length and count sessions of at least 20 minutes using foreach.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use a separate counter and convert the total to double before division.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-04-session-statistics.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-04-session-statistics
# Implement the exercise, then verify:
dotnet run --project .\CSharp\04-Loops
git diff -- CSharp/04-Loops/Program.cs
git add -- CSharp/04-Loops/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): count longer study sessions"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-04-session-statistics`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.04.Loops.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
int[] minutes = [15, 20, 25];
int total = 0;
for (int index = 0; index < minutes.Length; index++) {
  Console.WriteLine($"Session {index + 1}: {minutes[index]} minutes");
  total += minutes[index];
}
Console.WriteLine($"Total: {total} minutes");
int remaining = 2;
while (remaining > 0) {
  Console.WriteLine($"Practice rounds left: {remaining}");
  remaining--;
}
```

04-Loops exercise solution

Print the average session length and count sessions of at least 20 minutes using foreach.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
int longSessions = 0;
foreach (int duration in minutes) {
  if (duration >= 20) {
    longSessions++;
  }
}
```

```
Console.WriteLine($"Average: {(double)total / minutes.Length}");
Console.WriteLine($"Long sessions: {longSessions}");
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

05-Methods: Methods and reusable rules

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn parameters, return values, scope, and pure functions. Complete 04-Loops first.

Understand the idea

A method packages a reusable operation. Its signature names the return type, method, and parameters. Parameters are local to the method. return sends a result back to the caller. These examples use block-bodied methods under the Google style convention; C# also supports expression-bodied methods, which you may see in official documentation. static local functions do not capture surrounding variables. && means and. A pure function calculates a result from its inputs without changing external state, making it easy to test. Keep calculation separate from Console.WriteLine so callers can display, store, or compare the returned result.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\05-Methods
```

Expected result

```
10 + 5 = 15
20% of 50 = 10
Valid duration: True
```

Exercise

Write CalculateTotal(price, quantity). Reject negative prices and quantities below one. Verify 5 times 3 equals 15.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use ArgumentOutOfRangeException for invalid parameters.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-05-total-calculation.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-05-total-calculation
# Implement the exercise, then verify:
dotnet run --project .\CSharp\05-Methods
git diff -- CSharp/05-Methods/Program.cs
git add -- CSharp/05-Methods/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): validate total calculation inputs"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-05-total-calculation`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.05.Methods.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
Console.WriteLine($"10 + 5 = {Add(10, 5)}");
Console.WriteLine($"20% of 50 = {Percentage(50m, 20m)}");
Console.WriteLine($"Valid duration: {IsValidDuration(30)}");

// Parameters are inputs; the return type describes the output.
static int Add(int left, int right) {
  return left + right;
}

static decimal Percentage(decimal value, decimal percent) {
  return value * percent / 100m;
}

static bool IsValidDuration(int minutes) {
  return minutes > 0 && minutes <= 180;
}
```

05-Methods exercise solution

Write CalculateTotal(price, quantity). Reject negative prices and quantities below one. Verify 5 times 3 equals 15.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
static decimal CalculateTotal(decimal price, int quantity) {
```

```
  if (price < 0) {
    throw new ArgumentOutOfRangeException(nameof(price));
  }

  if (quantity < 1) {
    throw new ArgumentOutOfRangeException(nameof(quantity));
  }

  return price * quantity;
}
// Call from the top-level section: Console.WriteLine(CalculateTotal(5m, 3));
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

06-Collections: Lists, dictionaries, and null

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn choosing collections and handling missing values. Complete 05-Methods first.

Understand the idea

A List<T> can grow, unlike a fixed-size array. T is a type parameter; List<string> holds strings. A Dictionary<TKey,TValue> maps unique keys to values. Reading a missing key through the indexer throws; TryGetValue safely checks for it. Dictionary keys are case-sensitive for strings by default; you can provide StringComparer.OrdinalIgnoreCase if that fits your rule. A string? can contain text or null. The null-coalescing operator ?? supplies a fallback. Nullable reference analysis warns about possible null misuse at compile time, but validation is still needed for external data.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\06-Collections
```

Expected result

```
C#
PowerShell
.NET 10
C#: 45 minutes
Note: No note yet
```

Exercise

Add .NET 10 with 60 minutes. Print a total of 135 minutes, using a loop over dictionary values.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Values exposes the stored values; Sum is introduced later.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-06-topic-totals.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-06-topic-totals
# Implement the exercise, then verify:
dotnet run --project .\CSharp\06-Collections
git diff -- CSharp/06-Collections/Program.cs
git add -- CSharp/06-Collections/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): aggregate topic minutes"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-06-topic-totals`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.06.Collections.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
var topics = new List<string> { "C#", "PowerShell" };
topics.Add(".NET 10");
foreach (string topic in topics) {
  Console.WriteLine(topic);
}

var minutesByTopic = new Dictionary<string, int> {
  ["C#"] = 45,
  ["PowerShell"] = 30
};
if (minutesByTopic.TryGetValue("C#", out int minutes)) {
  Console.WriteLine($"C#: {minutes} minutes");
}

string? optionalNote = null;
Console.WriteLine($"Note: {optionalNote ?? "No note yet"}");
```

06-Collections exercise solution

Add .NET 10 with 60 minutes. Print a total of 135 minutes, using a loop over dictionary values.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
minutesByTopic[".NET 10"] = 60;
int total = 0;
```

```
foreach (int value in minutesByTopic.Values) {
  total += value;
}

Console.WriteLine($"Total: {total} minutes");
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

07-Classes: Classes, records, and encapsulation

Level: Intermediate | Suggested time: 45-60 minutes

Goal

Learn objects, constructors, properties, and records. Complete 06-Collections first.

Understand the idea

A class defines a type; new creates an instance. A constructor initializes it and can enforce rules before the object exists. Properties expose data. A get-only property is initialized by the constructor. A private set lets the class change a property while preventing callers from setting it directly. Complete describes why the state changes. sealed prevents inheritance when it is unnecessary. A record is useful for data values and provides value-based equality. Class instances ordinarily compare by identity. Program.cs contains top-level statements; each core type lives in its own named file under the Google style convention.

Step by step

1. Open Program.cs, StudySession.cs, and SessionSummary.cs. Read the comments before running them.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\07-Classes
```

Expected result

```
C#: 30 minutes; complete=True
Summary: C# / 30
```

Exercise

Add a CompletedAt property and set it when Complete is called. Preserve the first completion time if Complete is called again.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use a nullable DateTimeOffset and ??=.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-07-completion-time.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-07-completion-time
# Implement the exercise, then verify:
dotnet run --project .\CSharp\07-Classes
git diff -- CSharp/07-Classes/Program.cs CSharp/07-Classes/StudySession.cs
git add -- CSharp/07-Classes/Program.cs CSharp/07-Classes/StudySession.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): record first completion time"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Keep this commit limited to one exercise; explain the rule and the verification in its message body.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-07-completion-time`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.07.Classes.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
var session = new StudySession("C#", 30);
session.Complete();
Console.WriteLine($"{session.Topic}: {session.Minutes} minutes; complete={session.IsComplete}");
var summary = new SessionSummary(session.Topic, session.Minutes);
Console.WriteLine($"Summary: {summary.Topic} / {summary.Minutes}");
```

SessionSummary.cs

```
public sealed record SessionSummary(string Topic, int Minutes);
```

StudySession.cs

```
public sealed class StudySession {
  public string Topic {
    get;
  }
  public int Minutes {
    get;
  }
  public bool IsComplete {
    get; private set;
  }

  public StudySession(string topic, int minutes) {
    if (string.IsNullOrWhiteSpace(topic)) {
```

```
      throw new ArgumentException("Topic is required.", nameof(topic));
    }

    if (minutes <= 0) {
      throw new ArgumentOutOfRangeException(nameof(minutes));
    }

    Topic = topic;
    Minutes = minutes;
  }

  public void Complete() {
    IsComplete = true;
  }
}
```

07-Classes exercise solution

Add a CompletedAt property and set it when Complete is called. Preserve the first completion time if Complete is called again.

Suggested change

Add the property in StudySession.cs and replace that file's existing Complete method.

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
public DateTimeOffset? CompletedAt { get; private set; }

public void Complete() {
  CompletedAt ??= DateTimeOffset.UtcNow;
  IsComplete = true;
}
// Replace the existing Complete method inside StudySession.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

08-Interfaces: Interfaces and composition

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn contracts, polymorphism, and dependency injection. Complete 07-Classes first.

Understand the idea

An interface describes operations without choosing a specific implementation. ConsoleNotifier implements INotifier. A variable typed INotifier can refer to any implementation of that contract. StudyTracker receives its dependency through its constructor rather than constructing one internally. This is dependency injection; it works without a framework. The primary constructor syntax places the constructor parameters on the class declaration. Composition joins small objects to provide behavior. Replacing the notifier can change output without rewriting the tracker. This is a useful foundation for testing and for the .NET dependency injection lesson.

Step by step

1. Open Program.cs, INotifier.cs, ConsoleNotifier.cs, and StudyTracker.cs. Read their comments.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\08-Interfaces
```

Expected result

```
Completed: Interfaces
```

Exercise

Create a MemoryNotifier that stores messages in a List<string>. Call Finish twice and verify that it captured two messages.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Expose a read-only view or list property for inspection.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-08-memory-notifier.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-08-memory-notifier
# Implement the exercise, then verify:
dotnet run --project .\CSharp\08-Interfaces
git diff -- CSharp/08-Interfaces/Program.cs
git add -- CSharp/08-Interfaces/Program.cs CSharp/08-Interfaces/MemoryNotifier.cs
git diff --cached
git diff --cached --check
git commit -m "test(csharp): capture notifier messages in memory"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-08-memory-notifier`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

ConsoleNotifier.cs

```
public sealed class ConsoleNotifier : INotifier {
  public void Send(string message) {
    Console.WriteLine(message);
  }
}
```

CSharp.08.Interfaces.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

INotifier.cs

```
public interface INotifier {
  void Send(string message);
}
```

Program.cs

```
INotifier notifier = new ConsoleNotifier();
var tracker = new StudyTracker(notifier);
tracker.Finish("Interfaces");
```

StudyTracker.cs

```
public sealed class StudyTracker(INotifier notifier) {
  public void Finish(string topic) {
```

```
    notifier.Send($"Completed: {topic}");
  }
}
```

08-Interfaces exercise solution

Create a MemoryNotifier that stores messages in a List<string>. Call Finish twice and verify that it captured two messages.

Suggested change

Create MemoryNotifier.cs in this lesson folder:

```
public sealed class MemoryNotifier : INotifier {
  public List<string> Messages { get; } = [];

  public void Send(string message) {
    Messages.Add(message);
  }
}
```

Replace the top-level statements in Program.cs with:

```
var memory = new MemoryNotifier();
var tracker = new StudyTracker(memory);
tracker.Finish("C#");
tracker.Finish("PowerShell");
Console.WriteLine(memory.Messages.Count);
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

09-Linq: LINQ queries and lambdas

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn filtering, projection, aggregation, and deferred execution. Complete 08-Interfaces first.

Understand the idea

LINQ applies query operations to sequences. A lambda such as s => s.Minutes >= 30 is a small function used as a predicate. Where filters; Select transforms; OrderBy sorts; Sum aggregates; GroupBy partitions by a key. Many sequence operators are deferred: they run when the result is enumerated. ToList evaluates the query and stores a snapshot. If the source changes before enumeration, a deferred query can observe the change. Keep predicates free of side effects. LINQ over an in-memory list differs from database LINQ, where a provider may translate expressions into SQL.

Step by step

1. Open Program.cs and Session.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\09-Linq
```

Expected result

```
C#: 45
C#: 30
Total: 110
.NET 10 total: 15
C# total: 75
PowerShell total: 20
```

Exercise

Find the average session length and create a list of distinct topic names. Expect 27.5 minutes and three topics.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Use Average, Select, and Distinct; format numbers invariantly if necessary.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-09-topic-summary.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-09-topic-summary
# Implement the exercise, then verify:
dotnet run --project .\CSharp\09-Linq
git diff -- CSharp/09-Linq/Program.cs
git add -- CSharp/09-Linq/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): summarize distinct study topics"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-09-topic-summary`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.09.Linq.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
var sessions = new List<Session>
{
    new("C#", 30), new("PowerShell", 20), new("C#", 45), new(".NET 10", 15)
};
var longer = sessions.Where(s => s.Minutes >= 30).OrderByDescending(s => s.Minutes).ToList();
foreach (var session in longer) {
  Console.WriteLine($"{session.Topic}: {session.Minutes}");
}

Console.WriteLine($"Total: {sessions.Sum(s => s.Minutes)}");
foreach (var group in sessions.GroupBy(s => s.Topic).OrderBy(g => g.Key, StringComparer.Ordinal)) {
  Console.WriteLine($"{group.Key} total: {group.Sum(s => s.Minutes)}");
}
```

Session.cs

```
public sealed record Session(string Topic, int Minutes);
```

09-Linq exercise solution

Find the average session length and create a list of distinct topic names. Expect 27.5 minutes and three topics.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
Console.WriteLine(sessions.Average(s => s.Minutes));
var names = sessions.Select(s => s.Topic).Distinct().ToList();
Console.WriteLine($"Topics: {names.Count}");
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

10-Files: Files, JSON, and exceptions

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn serialization, paths, recoverable errors, and side effects. Complete 09-Linq first.

Understand the idea

Serialization converts an object into a representation such as JSON. Deserialization reconstructs an object from that representation. System.Text.Json is included with .NET. Path.Combine builds platform-appropriate paths. Directory.CreateDirectory is safe when the directory already exists. Writing a file changes persistent state, so this example uses a dedicated output directory. A try block surrounds operations that can fail; catch handles specific exception types. Exceptions outside these types still surface. Do not silently ignore errors: the example writes an error and sets a nonzero exit code. Deserialized values still need validation before business use.

Step by step

1. Open Program.cs and Session.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\10-Files
```

Expected result

```
Saved and loaded 2 sessions.
```

Exercise

Read a separate missing file inside a try block and show a friendly message. Do not delete the sample file.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

FileNotFoundException is a more specific IOException.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-10-missing-file.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-10-missing-file
# Implement the exercise, then verify:
dotnet run --project .\CSharp\10-Files
git diff -- CSharp/10-Files/Program.cs
git add -- CSharp/10-Files/Program.cs
git diff --cached
git diff --cached --check
git commit -m "fix(csharp): handle missing study files"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-10-missing-file`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.10.Files.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
using System.Text.Json;

// Output belongs to this lesson, regardless of the terminal's current directory.
string output = Path.GetFullPath(
    Path.Combine(AppContext.BaseDirectory, "..", "..", "..", "output"));
Directory.CreateDirectory(output);
string path = Path.Combine(output, "sessions.json");
var original = new List<Session> { new("C#", 30), new("PowerShell", 20) };
try {
  var options = new JsonSerializerOptions { WriteIndented = true };
  File.WriteAllText(path, JsonSerializer.Serialize(original, options));
  var loaded = JsonSerializer.Deserialize<List<Session>>(File.ReadAllText(path))
      ?? throw new InvalidDataException("The JSON contained null.");
  Console.WriteLine($"Saved and loaded {loaded.Count} sessions.");
} catch (IOException error) {
  Console.Error.WriteLine($"File error: {error.Message}");
  Environment.ExitCode = 1;
} catch (JsonException error) {
  Console.Error.WriteLine($"JSON error: {error.Message}");
  Environment.ExitCode = 1;
}
```

Session.cs

```
public sealed record Session(string Topic, int Minutes);
```

10-Files exercise solution

Read a separate missing file inside a try block and show a friendly message. Do not delete the sample file.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
try {
  File.ReadAllText(Path.Combine(output, "missing.json"));
} catch (FileNotFoundException) {
  Console.WriteLine("No saved file exists yet.");
}
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

11-Async: Async, await, and cancellation

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn tasks, overlapping waits, cancellation, and disposal. Complete 10-Files first.

Understand the idea

A Task represents an operation that may finish later. async enables await inside a method. await waits without synchronously blocking the calling thread. Async is mainly useful for I/O; it does not automatically make CPU-heavy work faster or create a new thread. Both operations start before WhenAll is awaited, so their delays overlap. Their log order can vary. A CancellationToken cooperatively requests cancellation; the operation must observe it. using disposes the cancellation source. Avoid .Result and .Wait when asynchronous calling code can await instead. Catch cancellation separately so a requested stop is not confused with an unexpected failure.

Step by step

1. Open Program.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\11-Async
```

Expected result

```
Prepared: C#
Prepared: PowerShell
Planned minutes: 50
(The first two lines may appear in either order.)
```

Exercise

Change the timeout to 10 milliseconds. Predict what happens, then run it. Restore the timeout and explain the difference.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

The simulated delay is 100 milliseconds and receives the token.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-11-cancellation.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-11-cancellation
# Implement the exercise, then verify:
dotnet run --project .\CSharp\11-Async
git diff -- CSharp/11-Async/Program.cs
git add -- CSharp/11-Async/Program.cs
git diff --cached
git diff --cached --check
git commit -m "test(csharp): verify planning cancellation"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-11-cancellation`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.11.Async.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
using var cancellation = new CancellationTokenSource(TimeSpan.FromSeconds(2));
try {
  Task<int> first = PracticeAsync("C#", 30, cancellation.Token);
  Task<int> second = PracticeAsync("PowerShell", 20, cancellation.Token);
  int[] minutes = await Task.WhenAll(first, second);
  Console.WriteLine($"Planned minutes: {minutes.Sum()}");
} catch (OperationCanceledException) {
  Console.WriteLine("Practice planning was cancelled.");
}
static async Task<int> PracticeAsync(string topic, int minutes, CancellationToken token) {
  // Simulate waiting for I/O; async does not inherently create a new thread.
  await Task.Delay(100, token);
  Console.WriteLine($"Prepared: {topic}");
  return minutes;
}
```

11-Async exercise solution

Change the timeout to 10 milliseconds. Predict what happens, then run it. Restore the timeout and explain the difference.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
using var cancellation = new CancellationTokenSource(TimeSpan.FromMilliseconds(10));
```

```
// Replace the original source declaration. Expected:
// Practice planning was cancelled.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.

12-Capstone: A persistent study tracker and C# 14

Level: Advanced within this course | Suggested time: 60-90 minutes

Goal

Learn combining parsing, collections, classes, json, and modern c#. Complete 11-Async first.

Understand the idea

The capstone is a command-line program with persistent data. First it loads the log, then selects a command, applies validation, and saves only after a valid change. The field keyword in the Topic property is a C# 14 feature: the compiler supplies the storage while the accessor enforces a rule. init allows assignment during initialization. JsonConstructor tells the serializer which constructor to use. A temporary sibling file reduces the risk of truncating a good log if serialization fails. This remains a single-user teaching application: simultaneous processes can overwrite each other's updates. Production storage needs concurrency control, backups, and schema migration.

Step by step

1. Open Program.cs and StudyEntry.cs in this lesson. Read the comments before running it.
2. Predict the main output on paper. Identify the inputs, the operation, and the result.
3. Run the commands below from the repository root. A comment beginning with # tells you which terminal to use.
4. Compare the output with the expected result. Explain any difference before changing the code.
5. Change one value and run again. Explain which lines caused the output to change.
6. Attempt the exercise before opening SOLUTION.md. Keep your first attempt so you can compare it.

Run

```
dotnet run --project .\CSharp\12-Capstone -- add CSharp 25
dotnet run --project .\CSharp\12-Capstone -- list
```

Expected result

```
Added: CSharp, 25 minutes
CSharp: 25 minutes
Entries: 1; total: 25 minutes
(Counts increase if you repeat add.)
```

Exercise

Add a summary command that prints total minutes per topic. Verify invalid input never changes the saved file.

Work in this lesson's source file. For multiple experiments, copy the example into this lesson's output directory rather than changing later lessons. Rebuild after C# edits.

Hint

Group entries by Topic and sum each group inside the switch.

Git checkpoint

Read the shared Git workflow first. Save the supplied baseline and finish the previous branch before starting. Use this branch for the exercise: feat/csharp-12-summary-command.

```
git switch main
git status --short
# If using GitHub: git fetch origin; git pull --ff-only origin main
git switch -c feat/csharp-12-summary-command
# Implement the exercise, then verify:
dotnet run --project .\CSharp\12-Capstone -- summary
git diff -- CSharp/12-Capstone/Program.cs
git add -- CSharp/12-Capstone/Program.cs
git diff --cached
git diff --cached --check
git commit -m "feat(csharp): add a per-topic summary command"
```

Stage only files you actually changed, and adapt the message to the change you made. If the exercise is just an experiment, record its result in a short lesson note and commit that note with a docs or test message instead. Review the branch's history and include validation evidence in its pull request.

For GitHub practice, publish deliberately with `git push -u origin feat/csharp-12-summary-command`, then open and review a PR to main. For local practice, switch to main and merge this branch using the shared guide. Choose one completion path. Record the resulting commit ID in Progress.md.

Explain it yourself

- What data enters this example, and what data leaves it?
- Which line implements the main rule?
- What would happen if a value were missing or invalid?
- Where would you place the rule if another caller needed it?

See SOLUTION.md after your attempt. The solution is a suggested modification to the working example, not a separate runnable project.

Complete example source

CSharp.12.Capstone.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

Program.cs

```
using System.Text.Json;

string folder = Path.GetFullPath(
    Path.Combine(AppContext.BaseDirectory, "..", "..", "..", "output"));
Directory.CreateDirectory(folder);
string path = Path.Combine(folder, "study-log.json");
string command = args.Length == 0 ? "list" : args[0].ToLowerInvariant();
try {
  List<StudyEntry> entries = File.Exists(path)
      ? JsonSerializer.Deserialize<List<StudyEntry>>(File.ReadAllText(path))
          ?? throw new InvalidDataException("The log contained null.")
      : [];
  switch (command) {
    case "add" when args.Length == 3:
      if (!int.TryParse(args[2], out int minutes) || minutes < 1 || minutes > 180) {
        throw new ArgumentException("Minutes must be between 1 and 180.");
      }

      var entry = new StudyEntry(args[1], minutes);
      entries.Add(entry);
      // Write to a temporary sibling before replacing the saved file.
      string temp = path + ".tmp";
      var options = new JsonSerializerOptions { WriteIndented = true };
      File.WriteAllText(temp, JsonSerializer.Serialize(entries, options));
      File.Move(temp, path, overwrite: true);
      Console.WriteLine($"Added: {entry.Topic}, {entry.Minutes} minutes");
      break;
    case "list":
      foreach (var item in entries) {
        Console.WriteLine($"{item.Topic}: {item.Minutes} minutes");
      }
```

```
      Console.WriteLine($"Entries: {entries.Count}; total: {entries.Sum(e => e.Minutes)} minutes");
      break;
    default:
      throw new ArgumentException("Usage: list | add <topic> <minutes>");
  }
} catch (Exception error) when (
    error is IOException or JsonException or ArgumentException or InvalidDataException) {
  Console.Error.WriteLine(error.Message);
  Environment.ExitCode = 1;
}
```

StudyEntry.cs

```
public sealed class StudyEntry {
  // C# 14: field refers to the compiler-created backing field.
  public string Topic {
    get;
    init => field = string.IsNullOrWhiteSpace(value)
        ? throw new ArgumentException("Topic is required.")
        : value.Trim();
  }
  public int Minutes {
    get; init;
  }

  [System.Text.Json.Serialization.JsonConstructor]
  public StudyEntry(string topic, int minutes) {
    if (minutes is < 1 or > 180) {
      throw new ArgumentOutOfRangeException(nameof(minutes));
    }

    Topic = topic;
    Minutes = minutes;
  }
}
```

12-Capstone exercise solution

Add a summary command that prints total minutes per topic. Verify invalid input never changes the saved file.

Suggested change

Apply this snippet in the location described by the lesson or the comments. If it replaces a method or variable declaration, remove the original definition first.

```
  case "summary":
    foreach (var group in entries.GroupBy(e => e.Topic).OrderBy(g => g.Key)) {
      Console.WriteLine($"{group.Key}: {group.Sum(e => e.Minutes)} minutes");
    }

    break;
// Insert before default in the existing switch.
```

Verify

Run the lesson again using its README commands. Check the exercise's stated result and one boundary or invalid case. An error for invalid input can be the correct result; explain why before continuing.

Write one sentence in Progress.md describing why the change works.